

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение
высшего образования
«САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
АЭРОКОСМИЧЕСКОГО ПРИБОРОСТРОЕНИЯ»

КАФЕДРА №33

ОТЧЕТ ЗАЩИЩЕН С ОЦЕНКОЙ_____

ПРЕПОДАВАТЕЛЬ

старший преподаватель

К.А. Жиданов

должность, уч. степень, звание

подпись, дата

инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ VIBE-CODE

по курсу: Технологии и методы программирования

СТУДЕНТ ГР. № 3333
номер группы

подпись, дата

А. П. Кретов
инициалы, фамилия

Санкт-Петербург
2025

ВВЕДЕНИЕ

В условиях современной цифровой среды эффективное управление временем и задачами приобретает особую актуальность. Одним из наиболее востребованных инструментов для повышения личной продуктивности остаются списки дел (To-Do Lists), позволяющие структурировать задачи, отслеживать их выполнение и планировать рабочий процесс.

В рамках данной работы был разработан простой и удобный веб-сервис для ведения списка задач, реализованный с использованием платформы Node.js, а также Telegram-бот, предоставляющий пользователю возможность получать актуальный список задач прямо в мессенджере. Такое сочетание традиционного веб-интерфейса и интеграции с Telegram обеспечивает гибкость и мобильность — пользователь может работать со своими задачами как с компьютера, так и со смартфона.

Целью проекта является создание минималистичного и надёжного инструмента управления задачами, ориентированного на простоту взаимодействия и доступность.

Для достижения поставленной цели были реализованы следующие ключевые задачи:

разработан веб-сервер на Node.js, позволяющий добавлять, редактировать и удалять задачи с сохранением данных в локальный JSON-файл;

создан интуитивно понятный веб-интерфейс для управления списком дел;

реализована интеграция с Telegram посредством бота, отображающего актуальный список задач по запросу пользователя.

Проект демонстрирует базовые навыки веб-разработки с использованием Node.js, работу с асинхронными операциями, обработку HTTP-запросов, а также взаимодействие с внешними API, в частности с Telegram Bot API.

В перспективе данный проект может быть доработан и расширен: возможна реализация поддержки нескольких пользователей, добавление полноценной системы авторизации, использование баз данных для хранения информации и внедрение дополнительных функций для более гибкого управления задачами.

СТРУКТУРА ПРОЕКТА

Структура проекта показана на Рисунке 1.

Имя	Тип	Размер
node_modules	Папка с файлами	
public	Папка с файлами	
app.js	файл JavaScript	5 КБ
package.json	Файл "JSON"	1 КБ
package-lock.json	Файл "JSON"	153 КБ
todos.db	SQLite	12 КБ

Рисунок 1 – Структура проекта

КОД ПРОЕКТА

Код и описание файла app.js:

Код app.js:

```
const express = require('express');
const bodyParser = require('body-parser');
const sqlite3 = require('sqlite3').verbose();
const TelegramBot = require('node-telegram-bot-api');
const basicAuth = require('basic-auth');

// Конфигурация
const PORT = 3000;
const TELEGRAM_BOT_TOKEN =
'8173296673:AAE5IIUsNXuQIBYZ3rSVbltrT2q2WrWPNxI';
const ADMIN_CREDENTIALS = { username: 'admin', password: '12345' };
```

```

// Инициализация приложения
const app = express();
const bot = new TelegramBot(TELEGRAM_BOT_TOKEN, {polling: true});

// Инициализация базы данных
const db = new sqlite3.Database('./todos.db', (err) => {
  if (err) {
    console.error('Ошибка при подключении к базе данных:',
err.message);
  } else {
    console.log('Подключено к базе данных SQLite');
    db.run(`CREATE TABLE IF NOT EXISTS todos (
      id INTEGER PRIMARY KEY AUTOINCREMENT,
      task TEXT NOT NULL,
      completed BOOLEAN DEFAULT 0,
      created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    )`);
  }
});

// Middleware для базовой авторизации
const auth = (req, res, next) => {
  const user = basicAuth(req);
  if (!user || user.name !== ADMIN_CREDENTIALS.username || user.pass !==
ADMIN_CREDENTIALS.password) {
    res.set('WWW-Authenticate', 'Basic realm=Authorization
Required');
    return res.status(401).send('Требуется авторизация');
  }
  next();
};

app.use(bodyParser.urlencoded({ extended: true }));
app.use(bodyParser.json());
app.use(express.static('public'));

// Обработчик команды /list в Telegram
bot.onText(/\/list/, (msg) => {
  const chatId = msg.chat.id;

  db.all('SELECT * FROM todos ORDER BY created_at DESC', [], (err,
rows) => {
    if (err) {
      bot.sendMessage(chatId, 'Ошибка при получении списка
задач');
      return;
    }
  }
});

```

```

    if (rows.length === 0) {
        bot.sendMessage(chatId, 'Список задач пуст');
        return;
    }

    let message = ' Список задач:\n\n';
    rows.forEach((task, index) => {
        const status = task.completed ? '✓' : ' ';
        message += `${index + 1}. ${status} ${task.task}\n`;
    });

    bot.sendMessage(chatId, message);
});

// Защищенные маршруты API
// Получить все задачи
app.get('/api/todos', auth, (req, res) => {
    db.all('SELECT * FROM todos ORDER BY created_at DESC', [], (err,
rows) => {
        if (err) {
            res.status(500).json({ error: err.message });
            return;
        }
        res.json(rows);
    });
});

// Добавить новую задачу
app.post('/api/todos', auth, (req, res) => {
    const { task } = req.body;
    if (!task) {
        return res.status(400).json({ error: 'Task is required' });
    }

    db.run('INSERT INTO todos (task) VALUES (?)', [task], function(err)
{
        if (err) {
            res.status(500).json({ error: err.message });
            return;
        }

        res.json({
            id: this.lastID,
            task,
            completed: 0

```

```

    });
  });
});

// Обновить статус задачи
app.put('/api/todos/:id', auth, (req, res) => {
  const { id } = req.params;
  const { completed } = req.body;

  db.run('UPDATE todos SET completed = ? WHERE id = ?', [completed ?
1 : 0, id], function(err) {
    if (err) {
      res.status(500).json({ error: err.message });
      return;
    }
    res.json({ success: true });
  });
});

// Удалить задачу
app.delete('/api/todos/:id', auth, (req, res) => {
  const { id } = req.params;

  db.run('DELETE FROM todos WHERE id = ?', [id], function(err) {
    if (err) {
      res.status(500).json({ error: err.message });
      return;
    }
    res.json({ success: true });
  });
});

// Статический HTML файл (требуется авторизация)
app.get('/', auth, (req, res) => {
  res.sendFile(__dirname + '/public/index.html');
});

// Запуск сервера
app.listen(PORT, () => {
  console.log(Сервер запущен на http://localhost:${PORT});
  console.log(Для доступа используйте логин:
${ADMIN_CREDENTIALS.username} и пароль: ${ADMIN_CREDENTIALS.password});
});

```

Что делает этот код?

1. Импортирует нужные модули:
 - http для создания сервера; ◦ fs и path для работы с файлами; ◦ url и querystring для обработки URL и данных из POST-запросов; ◦ а также storage.js — модуль для чтения/записи задач в файл.
2. Определяет порт для запуска сервера — 3000.
3. Функция getHtmlRows принимает массив задач и формирует из них HTML-строки для таблицы.
4. Функция handleRequest — главный обработчик всех HTTP-запросов к серверу:
 - При GET-запросе на / читает список задач из файла, вставляет их в шаблон index.html и отправляет результат клиенту.
 - При POST-запросах на /add, /delete, /edit обрабатывает соответственно добавление, удаление и редактирование задач, обновляет файл с задачами и перенаправляет обратно на главную страницу.
 - Для всех остальных путей возвращает ошибку 404.
5. Запускает сервер, который слушает порт 3000 и выводит сообщение об успешном старте.

код html:

```
<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Менеджер задач</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      max-width: 600px;
      margin: 0 auto;
      padding: 20px;
    }
    h1 {
      text-align: center;
      color: #333;
    }
    #task-form {
      display: flex;
```

```

    margin-bottom: 20px;
}
#task-input {
    flex: 1;
    padding: 10px;
    font-size: 16px;
}
button {
    padding: 10px 15px;
    background-color: #4CAF50;
    color: white;
    border: none;
    cursor: pointer;
}
button:hover {
    background-color: #45a049;
}
ul {
    list-style-type: none;
    padding: 0;
}
li {
    padding: 10px;
    background-color: #f9f9f9;
    margin-bottom: 5px;
    border-radius: 4px;
    display: flex;
    align-items: center;
}
.completed {
    text-decoration: line-through;
    color: #888;
}
.task-text {
    flex: 1;
    margin-left: 10px;
}
.delete-btn {
    background-color: #f44336;
    margin-left: 10px;
}

```



```

        .delete-btn:hover {
            background-color: #d32f2f;
        }
    </style>
</head>
<body>
    <h1>Мой Список Дел</h1>
    <form id="task-form">
        <input type="text" id="task-input" placeholder="Добавить новую задачу..."
required>
        <button type="submit">Добавить</button>
    </form>
    <ul id="task-list"></ul>

    <script>
        document.addEventListener('DOMContentLoaded', function() {
            const taskForm = document.getElementById('task-form');
            const taskInput = document.getElementById('task-input');
            const taskList = document.getElementById('task-list');

            // Загрузка задач при загрузке страницы
            loadTasks();

            // Добавление новой задачи
            taskForm.addEventListener('submit', function(e) {
                e.preventDefault();
                const taskText = taskInput.value.trim();
                if (taskText) {
                    addTask(taskText);
                    taskInput.value = "";
                }
            });

            // Загрузка задач с сервера
            function loadTasks() {
                fetch('/api/todos')
                    .then(response => response.json())
                    .then(tasks => {
                        taskList.innerHTML = "";
                        tasks.forEach(task => {
                            addTaskToDOM(task);

```

```

        });
    });
}

// Добавление задачи на сервер
function addTask(taskText) {
    fetch('/api/todos', {
        method: 'POST',
        headers: {
            'Content-Type': 'application/json',
        },
        body: JSON.stringify({ task: taskText })
    })
    .then(response => response.json())
    .then(task => {
        addTaskToDOM(task);
    });
}

// Добавление задачи в DOM
function addTaskToDOM(task) {
    const li = document.createElement('li');
    if (task.completed) {
        li.classList.add('completed');
    }

    const checkbox = document.createElement('input');
    checkbox.type = 'checkbox';
    checkbox.checked = task.completed;
    checkbox.addEventListener('change', function() {
        updateTaskStatus(task.id, this.checked);
    });

    const span = document.createElement('span');
    span.classList.add('task-text');
    span.textContent = task.task;

    const deleteBtn = document.createElement('button');
    deleteBtn.classList.add('delete-btn');
    deleteBtn.textContent = 'Удалить';
    deleteBtn.addEventListener('click', function() {
        deleteTask(task.id, li);
    });
}

```

```

        li.appendChild(checkbox);
        li.appendChild(span);
        li.appendChild(deleteBtn);
        taskList.appendChild(li);
    }

    // Обновление статуса задачи
    function updateTaskStatus(id, completed) {
        fetch(/api/todos/${id}, {
            method: 'PUT',
            headers: {
                'Content-Type': 'application/json',
            },
            body: JSON.stringify({ completed })
        })
        .then(response => response.json())
        .then(() => {
            loadTasks(); // Перезагружаем список для обновления стилей
        });
    }

    // Удаление задачи
    function deleteTask(id, li) {
        fetch(/api/todos/${id}, {
            method: 'DELETE'
        })
        .then(response => response.json())
        .then(() => {
            li.remove();
        });
    }
}
});
</script>
</body>
</html>

```

ПРИМЕР РАБОТЫ ПРОГРАММЫ

1. Авторизация:

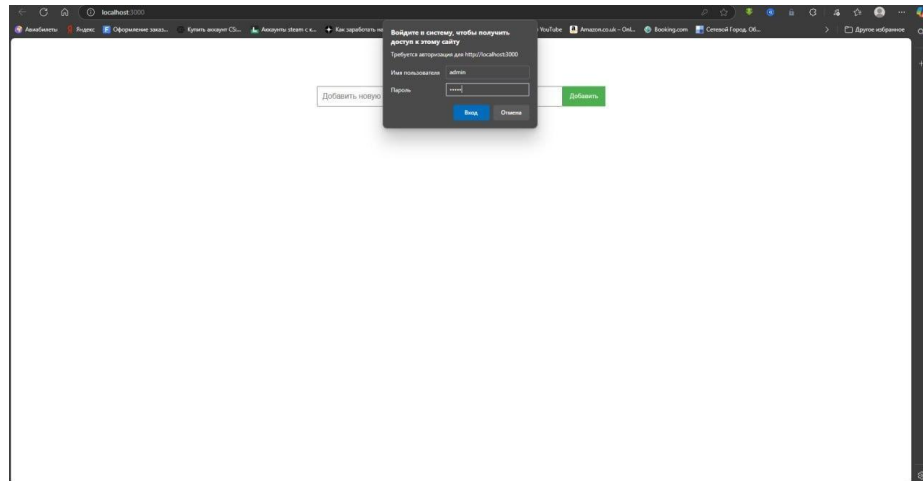


Рисунок 2 – Авторизация

2. Добавление задач

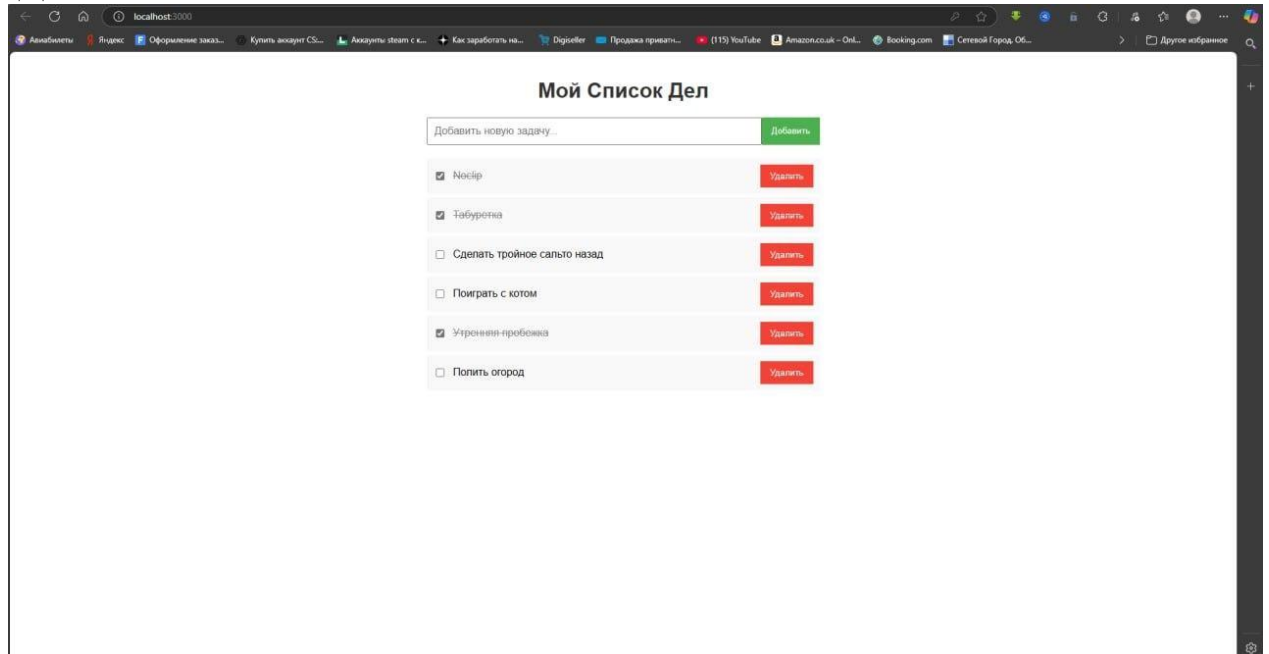


Рисунок 3 – Добавление задач

3. Пример работы с телеграмм-ботом. Вывод списка задач.

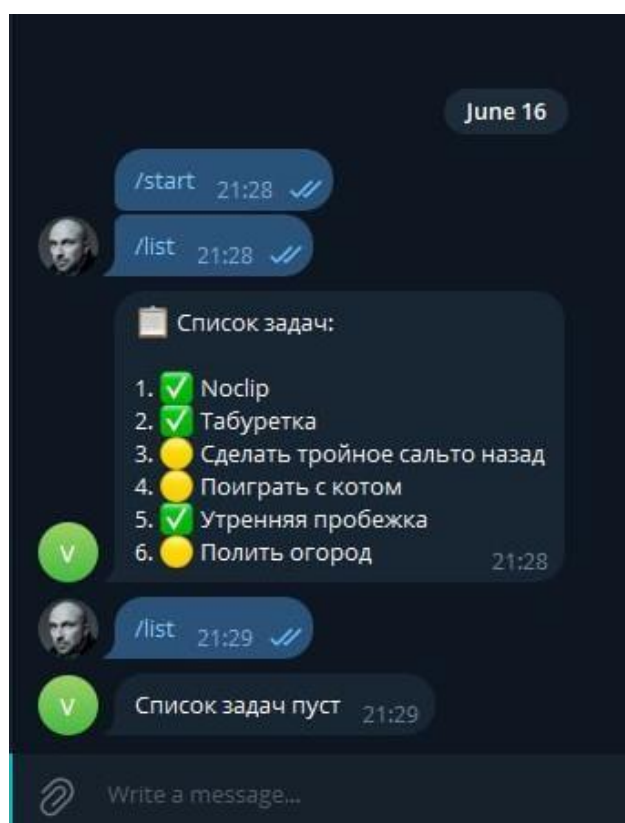


Рисунок 4 – Телеграмм бот

ПРИМЕР РАБОТЫ С НЕЙРОСЕТЬЮ

В процессе разработки проекта по созданию To-Do List с веб-интерфейсом и телеграм-ботом я активно использовал помощь нейросети ChatGPT, которая выступала в роли виртуального senior разработчика и наставника.

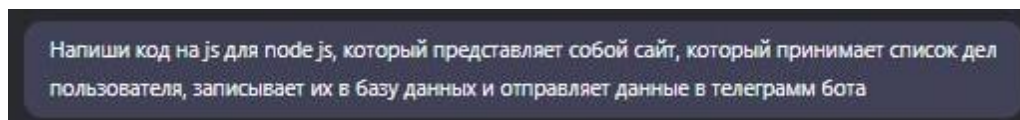
Что я просил нейросеть сделать

- Помочь с запуском и структурой Node.js сервера, принимающего запросы и хранящего задачи.
- Объяснить, как добавить и отредактировать файлы проекта.
- Реализовать простую авторизацию на сайте, чтобы при каждом заходе требовался ввод логина и пароля.
- Создать телеграм-бота, который выводит актуальный список задач, синхронизированный с сайтом.
- Предложить более надёжное решение для хранения задач с помощью записи в файл tasks.json.
- Написать полный и понятный код для сервера и бота.
- Объяснить структуру проекта и дать подробное описание кода для отчёта.
- Помочь выявить и устранить проблемы с авторизацией и синхронизацией данных.
- Предоставить полный обновлённый код сервера с авторизацией и хранением задач в файле.

Как нейросеть помогала

- Подробно объясняла логику и ход решения каждой задачи.
- Давала готовые решения с комментариями для быстрого внедрения.
- Предлагала оптимальные и простые способы реализации авторизации (базовая HTTP-авторизация).
- Помогала интегрировать хранение данных в файл для сохранности и синхронизации.
- Подсказывала, как исправить ошибки запуска и недостающие зависимости.
- Обеспечивала понимание структуры проекта и назначение каждого файла.
- Помогала формулировать отчёт и документировать проект.

Пример запроса:

A screenshot of a Telegram chat interface. It shows a single message in a dark-themed chat window. The message is in Russian and asks for a Node.js script that takes a list of tasks from a user, saves them to a database, and sends the data to a Telegram bot.

Напиши код на js для node js, который представляет собой сайт, который принимает список дел пользователя, записывает их в базу данных и отправляет данные в телеграмм бота

Рисунок 5 – Запрос

Удачный ответ:

```

javascript Copy Download

const express = require('express');
const bodyParser = require('body-parser');
const sqlite3 = require('sqlite3').verbose();
const TelegramBot = require('node-telegram-bot-api');
const basicAuth = require('basic-auth');

// Конфигурация
const PORT = 3000;
const TELEGRAM_BOT_TOKEN = 'YOUR_TELEGRAM_BOT_TOKEN';
const ADMIN_CREDENTIALS = { username: 'admin', password: '1234' };

// Инициализация приложения
const app = express();
const bot = new TelegramBot(TELEGRAM_BOT_TOKEN, {polling: true});

// Инициализация базы данных
const db = new sqlite3.Database('./todos.db', (err) => {
  if (err) {
    console.error('Ошибка при подключении к базе данных:', err.message);
  } else {
    console.log('Подключено к базе данных SQLite');
    db.run('CREATE TABLE IF NOT EXISTS todos (
      id INTEGER PRIMARY KEY AUTOINCREMENT,
      task TEXT NOT NULL,
      completed BOOLEAN DEFAULT 0,
      created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    )');
  }
});

// Middleware для базовой авторизации
const auth = (req, res, next) => {
  const user = basicAuth(req);
  if (!user || user.name !== ADMIN_CREDENTIALS.username || user.pass !== ADMIN_CREDENTIALS.password) {
    res.set('WWW-Authenticate', 'Basic realm=Authorization Required');
    return res.status(401).send('Требуется авторизация');
  }
  next();
};

app.use(bodyParser.urlencoded({ extended: true }));
app.use(bodyParser.json());
app.use(express.static('public'));

// Обработчик команды /list в Telegram
bot.onText(/\/list/, (msg) => {
  const chatId = msg.chat.id;

  db.all('SELECT * FROM todos ORDER BY created_at DESC', [], (err, rows) => {
    if (err) {
      bot.sendMessage(chatId, 'Ошибка при получении списка задач');
      return;
    }
  })
});

```

Рисунок 6 – Часть удачного ответа нейросети

ВЫВОД

В рамках работы над проектом был разработан полнофункциональный To-Do List с веб-интерфейсом и телеграм-ботом для оперативного доступа к списку задач. Основное внимание уделялось простоте, надёжности и удобству использования. Реализована базовая HTTP-авторизация: при каждом посещении сайта необходимо вводить логин и пароль, что обеспечивает базовую защиту без использования куки и сессий, сохраняя при этом приемлемый уровень безопасности.

В процессе разработки активно использовалась нейросеть ChatGPT, которая помогала в написании и отладке кода, объяснении логики и структурировании проекта. Это позволило ускорить процесс разработки, повысить качество решения и углубить понимание используемых технологий.

Таким образом, проект демонстрирует базовые принципы веб-разработки на Node.js, взаимодействие с внешними сервисами (такими как Telegram API), а также простую, но эффективную реализацию авторизации и хранения пользовательских данных.